

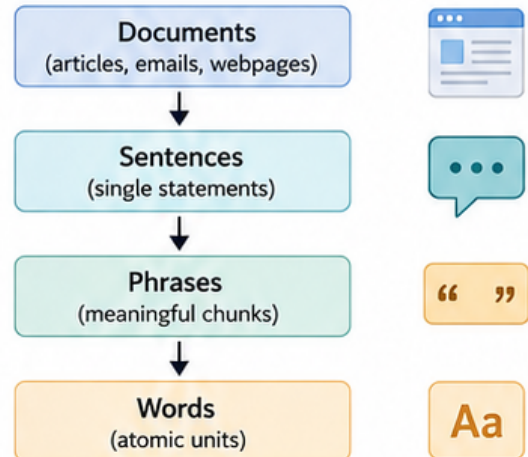
# Representation of Words

## Clear Notes – Part 1

### 1 Why do we need word representations?



- Machine learning algorithms need numbers, not raw words (text).
- Text can be represented at different levels:
  - Document** – an article, email, or webpage
  - Sentence** – a single statement
  - Phrase** – a meaningful chunk (e.g., “machine learning”)
  - Word** – the basic unit we start with



### 2 Traditional ways to represent text



#### • Bag of Words (BoW)

Represent a document using the counts of each word in the vocabulary.

Example vocabulary

|       |        |     |     |          |
|-------|--------|-----|-----|----------|
| apple | banana | car | dog | elephant |
|-------|--------|-----|-----|----------|

Document: “apple banana apple car”

→ [ 2 1 1 0 0 ]

#### • TF-IDF

Represent a document using term importance (higher weight for informative words).

Example vocabulary

|       |        |     |     |          |
|-------|--------|-----|-----|----------|
| apple | banana | car | dog | elephant |
|-------|--------|-----|-----|----------|

→ [ 0.81 0.57 0.25 0 0 ]



In both BoW and TF-IDF, each dimension corresponds to a vocabulary term.

### 3 Limitation of BoW / TF-IDF



- They ignore word order.
- Documents with the same words but different order get the same representation.
- They are sparse and high-dimensional: most values are 0.

#### Example

“apple banana” → [ 1 1 0 0 0 ]

“banana apple” → [ 1 1 0 0 0 ]

Same representation (order ignored)



### 4 One-Hot Encoding (OHE) – Introduction



- Each word is represented by a vector with one 1 and all other values 0.
- Useful for representing individual words in a vocabulary.

Example vocabulary (size = 5)

|   |        |      |    |     |
|---|--------|------|----|-----|
| a | andrew | good | is | boy |
|---|--------|------|----|-----|

One-hot vectors

a → [ 1, 0, 0, 0, 0 ]

andrew → [ 0, 1, 0, 0, 0 ]

good → [ 0, 0, 1, 0, 0 ]



Example sentence:

Andrew is a good boy

↓ ↓ ↓ ↓ ↓  
1 3 0 2 4

Using OHE, each word in the sentence is represented by its corresponding one-hot vector from the vocabulary.

# Representation of Words

## Clear Notes – Part 2

### 1 One-Hot Encoding – Worked Example

Vocabulary (size = 5)

|   |     |     |    |     |
|---|-----|-----|----|-----|
| a | cat | sat | on | the |
|---|-----|-----|----|-----|

#### One-hot vectors

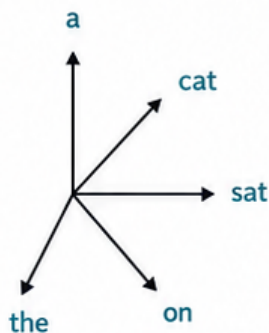
a → [ 1, 0, 0, 0, 0 ]

cat → [ 0, 1, 0, 0, 0 ]

sat → [ 0, 0, 1, 0, 0 ]

on → [ 0, 0, 0, 1, 0 ]

the → [ 0, 0, 0, 0, 1 ]



All vectors are orthogonal (90° apart).

#### Context example

Sentence: “a cat sat on the”

center word

|   |     |     |    |     |
|---|-----|-----|----|-----|
| a | cat | sat | on | the |
|---|-----|-----|----|-----|

Context words (window size = 2): [ a, cat, on, the ]

#### Context vectors (one-hot)

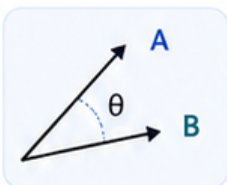
a → [ 1, 0, 0, 0, 0 ]

on → [ 0, 0, 0, 1, 0 ]

cat → [ 0, 1, 0, 0, 0 ]

the → [ 0, 0, 0, 0, 1 ]

### 2 Cosine Similarity



Cosine similarity between vectors A and B:

$$\cos(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

Tiny example with one-hot vectors

$\text{sim}(\text{a}, \text{cat}) = 0$

a = [ 1, 0, 0, 0, 0 ]

cat = [ 0, 1, 0, 0, 0 ]

Orthogonal vectors



Cosine similarity measures how similar two vectors point in the same direction.

### 3 Why One-Hot Is Not Enough



- All different words are equally far apart.
- Similarity between different words is often 0.
- It is sparse and high-dimensional.
- It does not capture meaning.



We need vectors whose distances actually mean something.

### 4 Quick Comparison



| Feature             | Integer Coding<br>(e.g., 1, 2, 3, ...) | One-Hot | Learned Embeddings<br>(dense vectors) |
|---------------------|----------------------------------------|---------|---------------------------------------|
| False order problem | ✗ Yes (bad)                            | ✓ No    | ✓ No                                  |
| Compactness         | ✓ Yes                                  | ✗ No    | ✓ Yes                                 |
| Distance meaning    | ✗ No                                   | ✗ No    | ✓ Yes                                 |
| Semantic similarity | ✗ No                                   | ✗ No    | ✓ Yes                                 |



# Representation of Words

## Clear Notes – Part 3

### 1 Distributed / Dense Word Representations

- Instead of huge sparse one-hot vectors, we learn dense, low-dimensional vectors called **embeddings**.
- These embeddings capture meaning.
- Words that are similar in usage tend to have similar (close) vectors.

Sparse One-Hot Vector  
(very high dimensional)

$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & \dots & 0 \end{bmatrix}$

Dim = 10,000s+  
(Very sparse)

Dense Embedding  
(low dimensional)

$\begin{bmatrix} 0.21 & -0.15 & 0.37 & 0.08 & -0.12 \end{bmatrix}$

Dim = 50 to 300  
(Dense)

### 2 Distributional Hypothesis

“Words that appear in similar contexts tend to have similar meanings.”

Intuition with examples:



I traveled by bus.



I traveled by car.

Both sentences occur in similar contexts (travel/transportation).



Because of this, the model learns vectors for “bus” and “car” that are close to each other in the vector space.

### 3 What Embeddings Capture



- Semantic similarity** – similar meanings have similar vectors.



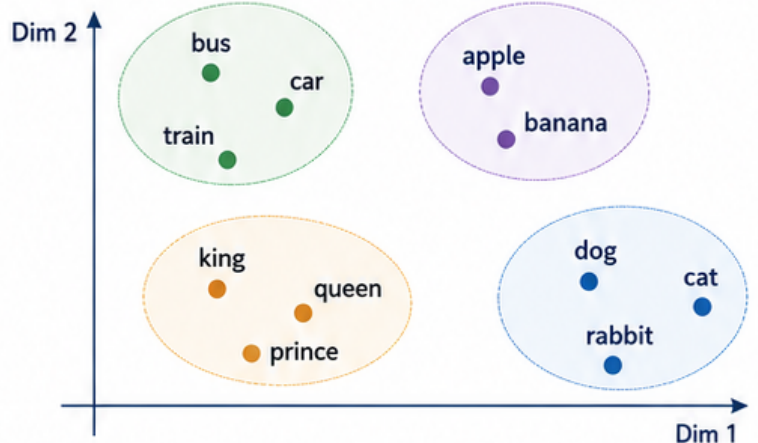
- Useful distances** – closer vectors mean more similar words.



- Meaningful neighborhoods** – words group into intuitive clusters.



- Lower dimensionality** – compact representations that still work well.



### 4 Vector Analogies

king – man + woman ≈ queen

Paris – France + Italy ≈ Rome

big – bigger + small ≈ smaller



Embeddings can capture relationships such as:

- Gender** ( $king - man + woman \approx queen$ )
  - Country-capital** ( $Paris - France + Italy \approx Rome$ )
  - Degree / size** ( $big - bigger + small \approx smaller$ )
- and many more!

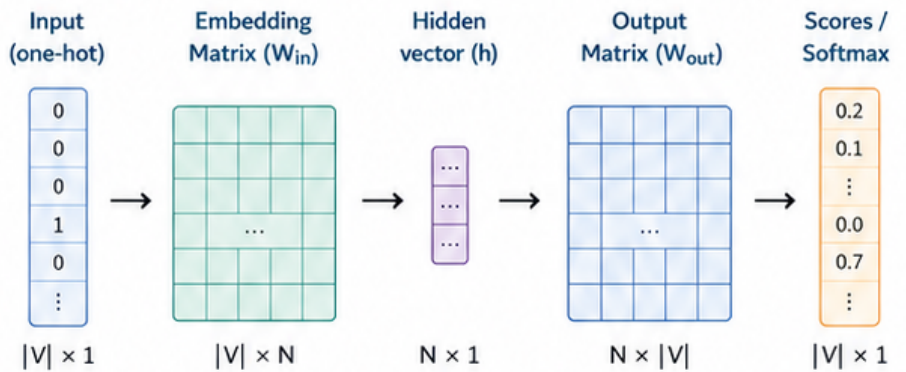
# Representation of Words

## Clear Notes – Part 4

### 1 Word2Vec Idea



- Word2Vec learns word embeddings from the context in which words appear.
- Two classic training setups: CBOW and Skip-gram.
- The learned rows of the input embedding matrix ( $W_{in}$ ) are the word embeddings.

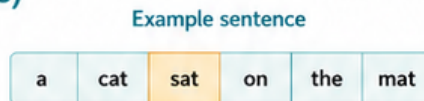


$V$  = vocabulary size,  $N$  = embedding dimension.

### 2 CBOW (Continuous Bag of Words)



- Use surrounding context words (bag) to predict the center word.
- We average the context word embeddings to form the hidden representation.



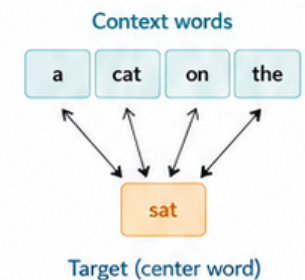
Window size = 2

$$h = \text{average}(E(a), E(cat), E(on), E(the))$$

$$= \frac{E(a) + E(cat) + E(on) + E(the)}{4}$$

Averaging the context embeddings yields the hidden vector  $h$  used to predict the center word "sat".

CBOW example (target = center word)

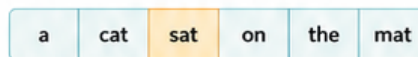


### 3 Skip-gram

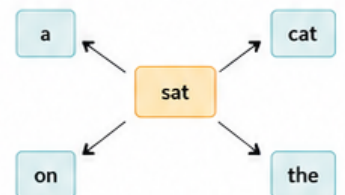


- Use the center word to predict surrounding context words.
- One center word generates multiple training targets (each context word).

Example sentence



Skip-gram example (center = "sat")



Predict surrounding context words

CBOW vs Skip-gram

CBOW

Context  $\rightarrow$  Center



Skip-gram

Center  $\rightarrow$  Context



### 4 Mini Training Example



Small corpus

$S_1$ : the cat sat on mat  
 $S_2$ : the dog sat on mat  
 $S_3$ : the cat ate fish

Vocabulary (unique words)



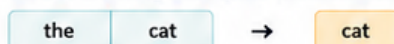
$V = 8$  words



After training, semantically related words (e.g., cat and dog) obtain similar embedding vectors.

CBOW training examples (window size = 2)

1



Context = [the, sat]  $\rightarrow$  predict center word "cat".

2



Context = [cat, on]  $\rightarrow$  predict center word "sat".